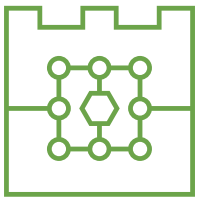


Politechnika Krakowska
im. Tadeusza Kościuszki



Wydział Informatyki
i Matematyki

Politechnika krakowska im. Tadeusza Kościuszki w Krakowie

Wydział Informatyki i Matematyki

Wykrywanie Zagrożeń i Reakcja na Incydenty

Analiza ruchu sieciowego w detekcji manipulacji komunikacją HTTPS w środowisku Mobile-to-Backend

Autor: Weronika Wojtas, Radosław Rolka
Studia: Informatyka-Cyberbezpieczeństwo, II stopień, I semestr

Kraków, 2026

Spis treści

1 Wprowadzenie **4**

2 Architektura środowiska **5**

3 Konfiguracja środowiska **7**
3.1 Uruchomienie kontenerów Docker 7
3.2 Uruchomienie emulatora Androida 7
3.3 Konfiguracja proxy i instalacja certyfikatu CA 9
3.4 Weryfikacja połączenia 9

4 Scenariusz 1: TLS Downgrade + HSTS **10**
4.1 Opis ataku 10
4.2 Uruchomienie 10
4.3 Wyniki 10
4.4 Wnioski 12

5 Scenariusz 2: Traffic Tampering **13**
5.1 Opis ataku 13
5.2 Uruchomienie 13
5.3 Wyniki 13
5.3.1 Kradzież danych logowania 13
5.3.2 Modyfikacja danych finansowych 14
5.4 Wnioski 16

6 Scenariusz 3: SSL Pinning Bypass **17**
6.1 Opis ataku 17
6.2 Uruchomienie 17
6.3 Wyniki 17
6.4 Wnioski 19

7 Podsumowanie **20**
7.1 Użyte technologie 20

Spis rysunków **21**

Spis tabel **22**

Rozdział 1

Wprowadzenie

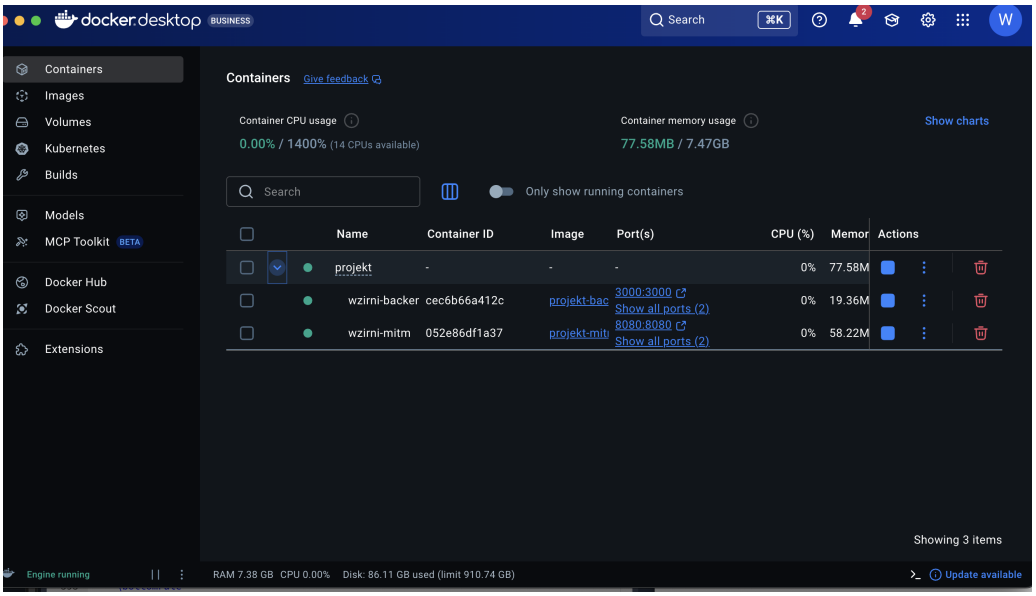
Celem projektu było stworzenie środowiska laboratoryjnego do analizy ruchu HTTPS między emulatorem Androida a serwerem backendowym z wykorzystaniem ataków MITM (*Man-in-the-Middle*). Zrealizowano trzy scenariusze:

1. **TLS Downgrade** — usunięcie nagłówka HSTS i podmiana linków `https://` na `http://`,
2. **Traffic Tampering** — przechwytywanie danych logowania i modyfikacja odpowiedzi serwera,
3. **SSL Pinning Bypass** — podmiana certyfikatu TLS przez mitmproxy.

Rozdział 2

Architektura środowiska

Środowisko składa się z trzech komponentów działających w izolowanej sieci Docker (`172.20.0.0/24`). Schemat architektury przedstawia rysunek 2.1.



Rysunek 2.1: Działające kontenery `wzirni-backend` i `wzirni-mitm`

```
macOS (host)                Docker (172.20.0.0/24)
+----------------+          +-------------+    +-------------+
| Android Studio |--------->|  mitmproxy  |--->|   Backend   |
|   Emulator     |  proxy   |   (MITM)    |    |  Node.js    |
|  API 28 (9.0)  | :8080    | 172.20.0.20 |    | 172.20.0.10 |
+----------------+          |  :8081 UI   |    |  :3443 HTTPS|
                            +-------------+    +-------------+
```

Listing 2.1: Schemat sieci środowiska laboratoryjnego

Poszczególne komponenty pełnią następujące role:

- **Backend Node.js** — serwer HTTPS (port 3443) i HTTP (port 3000) z endpointami `/api/health`, `/api/data`, `/api/login`; serwuje nagłówek HSTS i używa TLS 1.2+.
- **mitmproxy** — proxy przechwytujące ruch między emulatorem a backendem; każdy scenariusz uruchamia inny skrypt Pythona.

- **Emulator Android 9.0 (API 28)** — działa natywnie na macOS, skonfigurowany z proxy `10.0.2.2:8080` i certyfikatem CA mitmproxy zainstalowanym w system store.

Rozdział 3

Konfiguracja środowiska

3.1. Uruchomienie kontenerów Docker

Kontenery uruchamiane są poleceniami:

```
make build
make up
```

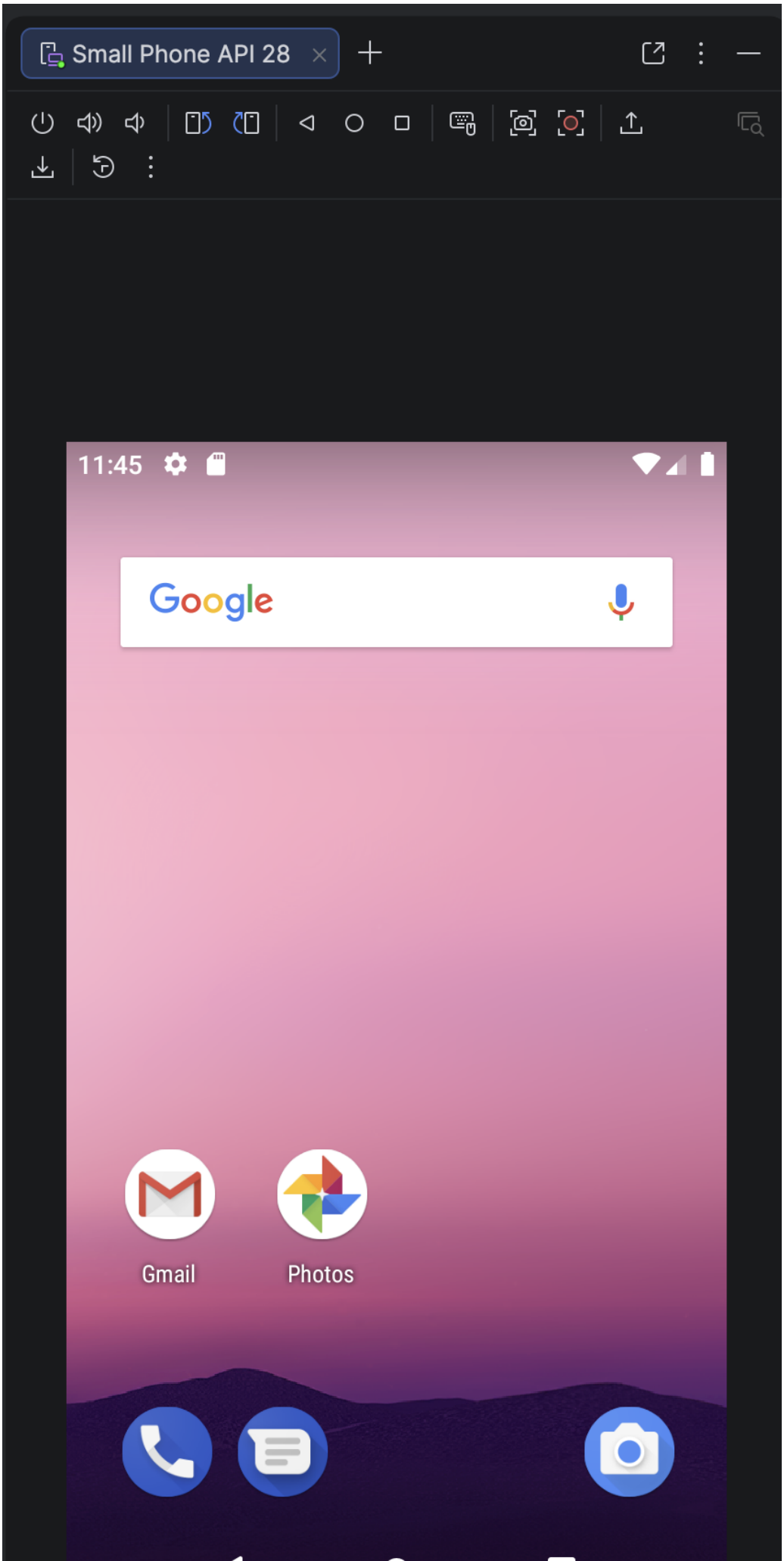
Listing 3.1: Uruchomienie środowiska Docker

3.2. Uruchomienie emulatora Androida

Emulator uruchamiany jest z flagą `-writable-system`, która umożliwia instalację certyfikatu CA do system store:

```
~/Library/Android/sdk/emulator/emulator @Small_Phone -writable-system &
adb root && adb disable-verity && adb reboot
adb wait-for-device && adb root && adb remount
```

Listing 3.2: Uruchomienie emulatora i przygotowanie systemu


Small Phone API 28
11:45
Google
Gmail
Photos

3.3. Konfiguracja proxy i instalacja certyfikatu CA

```
PATH="$PATH:$HOME/Library/Android/sdk/platform-tools" bash android/setup.sh
```

Listing 3.3: Uruchomienie skryptu konfiguracyjnego

Skrypt `setup.sh` automatycznie:

- ustawia proxy `10.0.2.2:8080` na emulatorze,
- eksportuje certyfikat CA z kontenera mitmproxy,
- instaluje certyfikat do `/system/etc/security/cacerts/` (wymaga root i flagi `-writable-system`).

```
→  PROJEKT git:(WZINRI_PROJECT) ✗ PATH="$PATH:$HOME/Library/Android/sdk/platform-tools" bash android/setup.sh
==============================================
 WZIRNI Android Emulator Setup (macOS)
 Android 9.0 / API 28
==============================================
[1/4] Czekam na gotowość emulatora ADB...
adbd is already running as root
      Próba wyłączenia verified boot (Android 9 / API 28)...
      System jest zapisywalny (remount succeeded).
[2/4] Ustawienie proxy -> 10.0.2.2:8080...
      Proxy ustawione (http_proxy + global_http_proxy).
[3/4] Eksport certyfikatu CA z Docker...
[4/4] Instalacja certyfikatu CA...
/tmp/mitmproxy-ca-cert.pem: 1 file pushed, 0 skipped. 1.6 MB/s (1172 bytes in 0.001s)
      SUKCES: Certyfikat zainstalowany w System Store.

==============================================
 Setup zakończony!
==============================================
 Android proxy:  10.0.2.2:8080
 mitmweb UI:     http://localhost:8081
 Backend HTTPS:  https://localhost:3443/api/health

 Test połączenia (otwiera przeglądarkę na emulatorze):
   adb shell am start -a android.intent.action.VIEW -d 'http://10.0.2.2:3000/api/health'

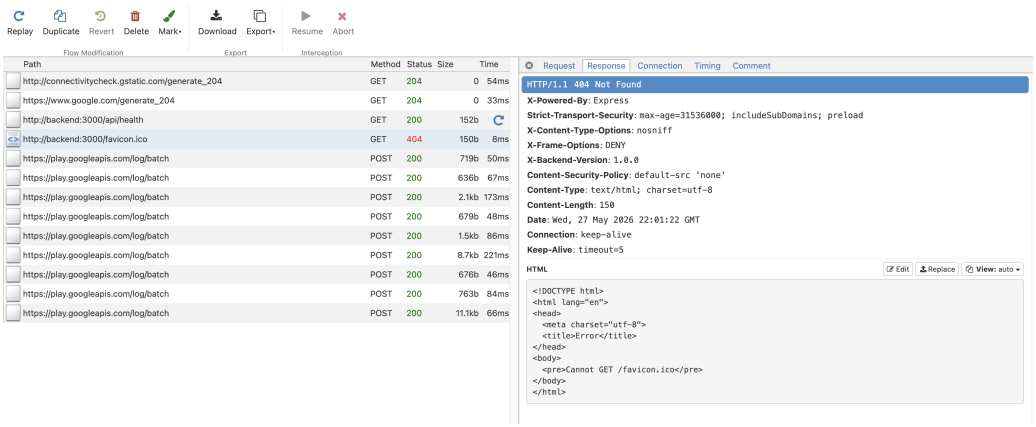
 Sprawdź przechwycony ruch w mitmweb: http://localhost:8081
==============================================
```

Rysunek 3.2: Wyjście skryptu `setup.sh` z komunikatem "Setup zakończony!"

3.4. Weryfikacja połączenia

```
adb shell am start -a android.intent.action.VIEW \
    -d 'http://10.0.2.2:3000/api/health'
```

Listing 3.4: Test połączenia emulatora z backendem



Rysunek 3.3: mitmweb UI (`http://localhost:8081`) z pierwszym przechwyconym żądaniem z emulatora

Rozdział 4

Scenariusz 1: TLS Downgrade + HSTS

4.1. Opis ataku

Skrypt `tls_downgrade.py` modyfikuje odpowiedzi serwera w trzech krokach:

1. usuwa nagłówek `Strict-Transport-Security` (HSTS), który wymuszałby użycie HTTPS przez rok,
2. zamienia wszystkie wystąpienia `https://` na `http://` w treści odpowiedzi JSON i HTML,
3. dodaje nagłówek `x-mitm-tls-downgrade: attempted` jako znacznik ingerencji.

4.2. Uruchomienie

```
make scenario-tls
bash demo.sh tls
```

Listing 4.1: Uruchomienie scenariusza TLS Downgrade

4.3. Wyniki

Poniżej zestawiono nagłówki odpowiedzi przed atakiem i po ataku.

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
api_url: "https://172.20.0.10:3443/api/data"
```

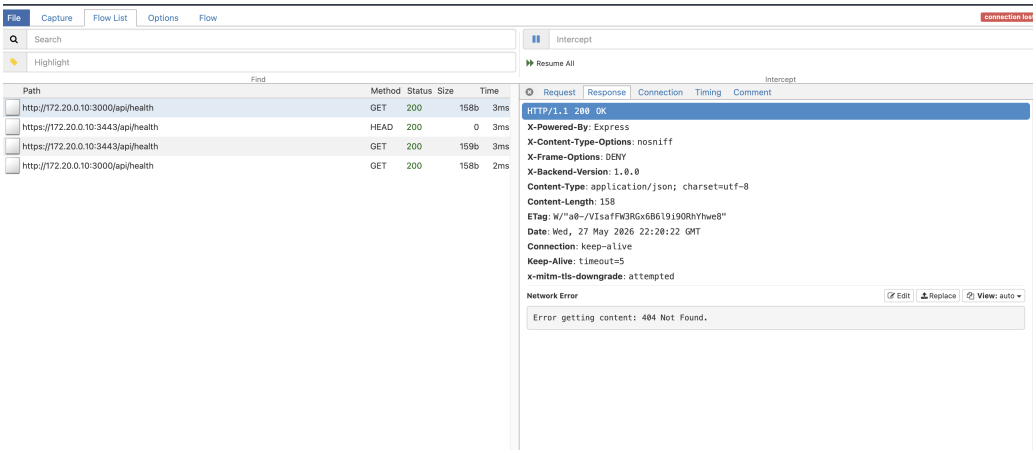
Listing 4.2: Odpowiedź serwera bez ataku

```
Strict-Transport-Security: <usuniety>
api_url: "http://172.20.0.10:3443/api/data"
x-mitm-tls-downgrade: attempted
```

Listing 4.3: Odpowiedź serwera po ataku TLS Downgrade



Rysunek 4.1: Wyjście `demo.sh tls` — porównanie nagłówków przed i po ataku



Rysunek 4.2: mitmweb UI — nagłówki odpowiedzi po modyfikacji (brak HSTS, obecny `x-mitm-tls-downgrade`)

4.4. Wnioski

Atak TLS Downgrade jest skuteczny w warstwie aplikacyjnej — mitmproxy usuwa mechanizm ochrony HSTS zanim dotrze do klienta. Ochroną jest preloading HSTS w przeglądarkach (lista Chromium HSTS Preload). Na Android 9 (API 28) aplikacje natywne domyślnie blokują ruch HTTP (`usesCleartextTraffic=false`), więc podmiana linków działa głównie w przeglądarce Chrome.

Rozdział 5

Scenariusz 2: Traffic Tampering

5.1. Opis ataku

Skrypt `traffic_tamper.py` wykonuje dwie operacje:

1. **Przechwytywanie danych logowania** — żądania `POST /api/login` są parsowane, a dane `username` i `password` zapisywane do pliku `/tmp/captured_credentials.log`.
2. **Modyfikacja odpowiedzi** — odpowiedź `GET /api/data` jest zmieniana: saldo `9999.99 PLN` → `0.01 PLN`, token zastępowany wartością `TAMPERED_TOKEN_BY_MITM`, dodawane pole `_mitm_note`.

Dodatkowo do odpowiedzi HTML wstrzykiwany jest JavaScript (listing 5.1).

```
<script>console.log("MITM injected script");</script>
```

Listing 5.1: Wstrzyknięty fragment JavaScript

5.2. Uruchomienie

```
make scenario-tamper
bash demo.sh tamper
```

Listing 5.2: Uruchomienie scenariusza Traffic Tampering

5.3. Wyniki

5.3.1. Kradzież danych logowania

Wysłano żądanie `POST /api/login` z danymi `{"username":"admin","password":"password123"}`. mitmproxy przechwycił i zapisał następujący wpis:

```
[CREDENTIAL-HARVEST] POST /api/login | user=admin | pass=password123
```

Listing 5.3: Przechwycone dane logowania w pliku logu

```
→  PROJEKT git:(WZINRI_PROJECT) ✗ make capture-logs
=== Traffic Tamper Log ===
docker exec wzirni-mitm cat /tmp/mitm_traffic.log 2>/dev/null || echo "(brak)"
[2026-05-27 22:26:03] [REQUEST] GET http://172.20.0.10:3000/api/health
[2026-05-27 22:26:03] [RESPONSE] GET http://172.20.0.10:3000/api/health -> HTTP 200
[2026-05-27 22:26:03] [CREDENTIAL-HARVEST] POST /api/login | user=admin | pass=password123
[2026-05-27 22:26:03] [REQUEST] POST https://172.20.0.10:3443/api/login
[2026-05-27 22:26:03] [RESPONSE] POST https://172.20.0.10:3443/api/login -> HTTP 200
[2026-05-27 22:26:03] [REQUEST] GET https://172.20.0.10:3443/api/data
[2026-05-27 22:26:03] [RESPONSE] GET https://172.20.0.10:3443/api/data -> HTTP 200
[2026-05-27 22:26:03] [TAMPER-RESP] /api/data | token: 99f46f61... -> TAMPERED | balance: 9999.99 -> 0.01

=== Captured Credentials ===
docker exec wzirni-mitm cat /tmp/captured_credentials.log 2>/dev/null || echo "(brak)"
2026-05-27T22:26:03.593097 | admin:password123

=== SSL Pinning Log ===
docker exec wzirni-mitm cat /tmp/ssl_pinning_log.log 2>/dev/null || echo "(brak)"
(brak)
```

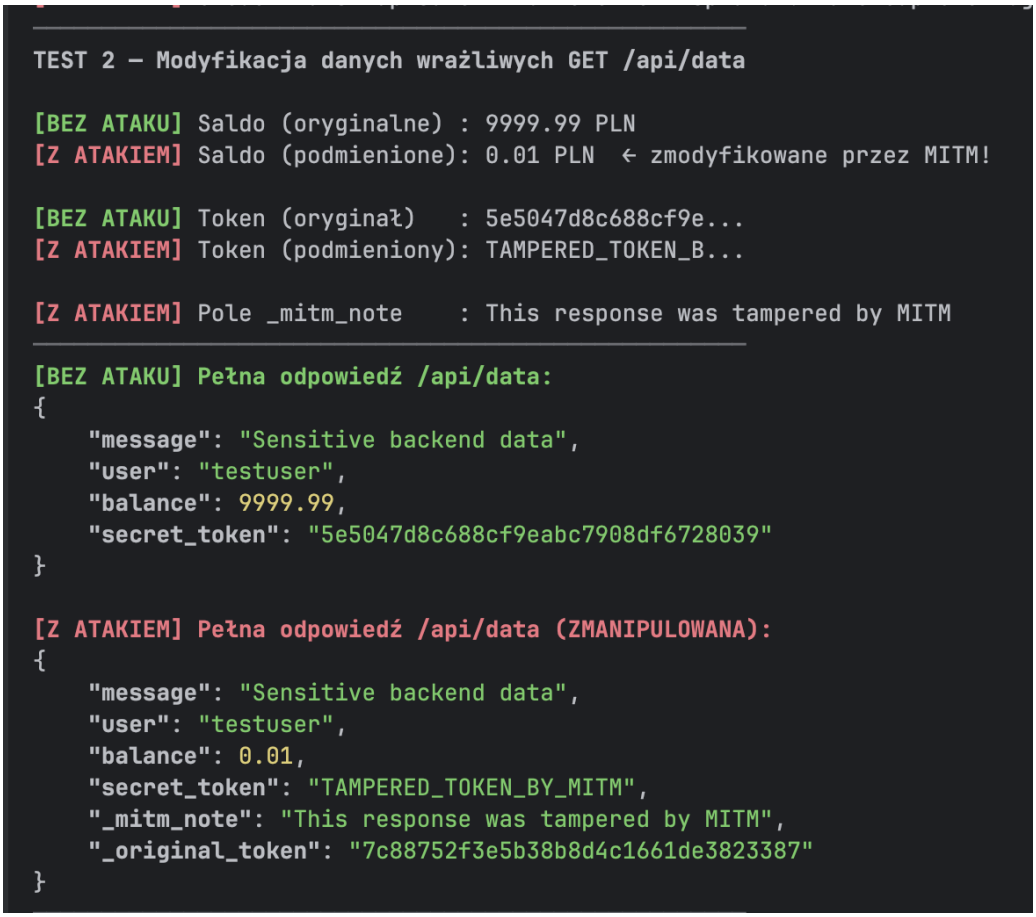
Rysunek 5.1: Wyjście `make capture-logs` z sekcją "Captured Credentials"

5.3.2. Modyfikacja danych finansowych

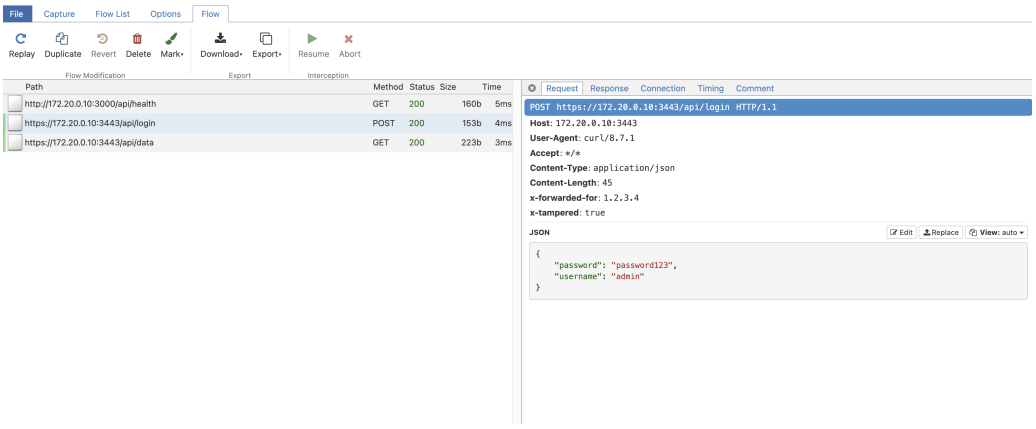
Tabela 5.1 zestawia wartości pól w odpowiedzi serwera przed atakiem i po ataku.

Tabela 5.1: Porównanie danych odpowiedzi `GET /api/data` przed atakiem i po ataku

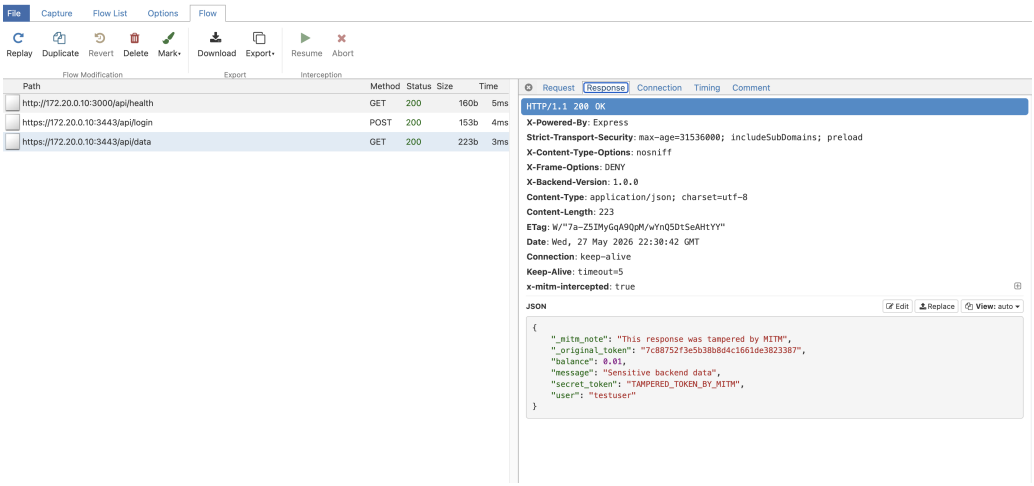
Pole	Oryginał	Po ataku
`balance`	`9999.99`	`0.01`
`secret_token`	`a3f8...` (losowy)	`TAMPERED_TOKEN_BY_MITM`
`_mitm_note`	(brak)	`This response was tampered by MITM`



Rysunek 5.2: Wyjście `demo.sh tamper` — porównanie salda przed atakiem i po ataku



Rysunek 5.3: mitmweb UI — żądanie `POST /api/login` z przechwycone danymi logowania (zakładka Request)



Rysunek 5.4: mitmweb UI — żądanie `GET /api/data` ze zmodyfikowaną odpowiedzią (zakładka Response)

5.4. Wnioski

Atak jest w pełni skuteczny, gdy klient nie weryfikuje certyfikatu serwera (flaga `-k` w curl lub zainstalowany CA mitmproxy). Dane przesyłane przez HTTPS nie są widoczne dla atakującego *bez* kontroli nad proxy. Ochroną jest weryfikacja integralności odpowiedzi po stronie klienta (np. sygnatury kryptograficzne) lub certificate pinning.

Rozdział 6

Scenariusz 3: SSL Pinning Bypass

6.1. Opis ataku

mitmproxy działa jako proxy TLS — nawiązuje dwa osobne połączenia:

- klient (emulator) ↔ mitmproxy: certyfikat wystawiony przez CA mitmproxy,
- mitmproxy ↔ backend: oryginalny certyfikat serwera.

Skrypt `ssl_pinning_bypass.py` loguje szczegóły handshake TLS i oznacza odpowiedzi nagłówkami `x-mitm-ssl-bypass: success` oraz `x-mitm-cert-replaced: mitmproxy-ca`.

Warunkiem powodzenia jest certyfikat CA mitmproxy zainstalowany w system trust store emulatora (wykonane przez `setup.sh`). Bez tego klient odrzuca certyfikat mitmproxy i połączenie jest zrywane.

6.2. Uruchomienie

```
make scenario-pinning
bash demo.sh pinning
```

Listing 6.1: Uruchomienie scenariusza SSL Pinning Bypass

6.3. Wyniki

```
issuer: CN=wzirni-backend (self-signed)
```

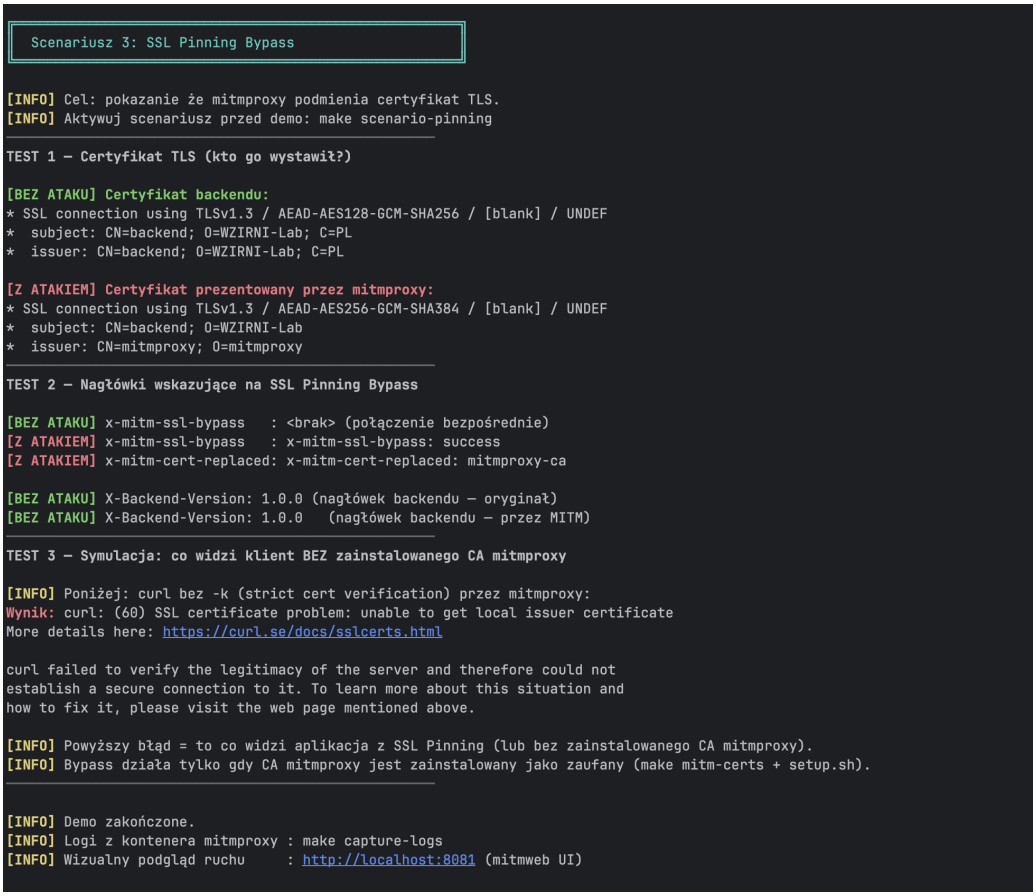
Listing 6.2: Certyfikat serwera bez ataku (połączenie bezpośrednie)

```
issuer: CN=mitmproxy, O=mitmproxy
x-mitm-ssl-bypass: success
x-mitm-cert-replaced: mitmproxy-ca
```

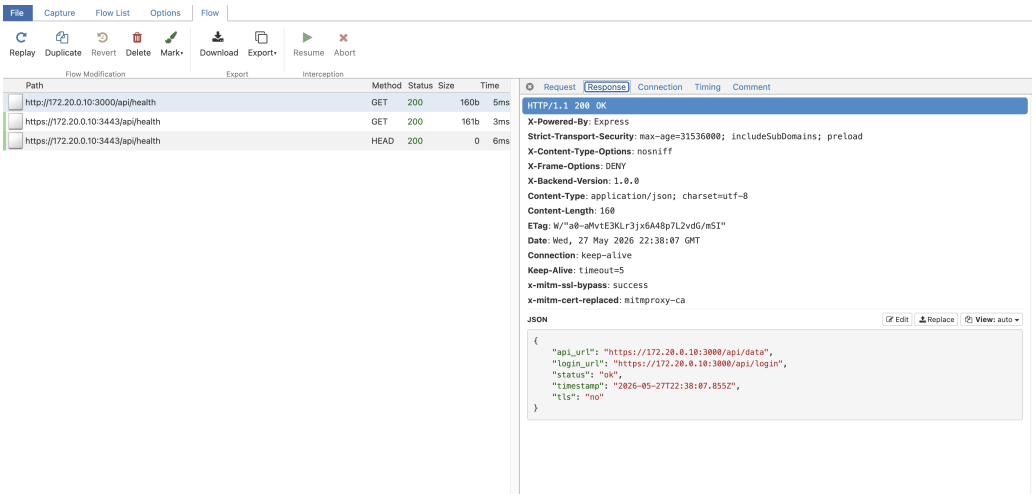
Listing 6.3: Certyfikat prezentowany przez mitmproxy

```
curl: (60) SSL certificate problem: unable to get local issuer certificate
```

Listing 6.4: Błąd klienta bez zainstalowanego CA mitmproxy



Rysunek 6.1: Wyjście `demo.sh pinning` — dane certyfikatu oryginalnego vs. certyfikatu mitmproxy



Rysunek 6.2: mitmweb UI — TLS handshake i nagłówki `x-mitm-ssl-bypass` w przechwyconej odpowiedzi

screenshots/13_android_ca_installed.png

[screenshot do uzupełnienia]

Rysunek 6.3: Certyfikat CA mitmproxy widoczny wśród zaufanych certyfikatów systemowych emulatora

6.4. Wnioski

SSL Pinning Bypass przez podmianę CA działa na urządzeniach z zainstalowanym certyfikatem atakującego w system trust store. Na Android 9+ (API 24+) certyfikaty użytkownika nie są ufane przez aplikacje — wymagana jest instalacja do system store z uprawnieniami root. Aplikacje z twardym pinningiem (NSC lub natywnym) wymagają bardziej zaawansowanych technik (Frida, modyfikacja APK).

Rozdział 7

Podsumowanie

Tabela 7.1 zestawia wyniki wszystkich trzech scenariuszy.

Tabela 7.1: Zestawienie wyników scenariuszy ataków MITM

Scenariusz	**Skuteczność**	**Warunek**	**Ochrona**
TLS Downgrade	Częściowa	Klient wysyła HTTP po downgrade	HSTS Preload, `usesCleartextTraffic=false`
Traffic Tampering	Pełna	CA mitmproxy zainstalowany	Weryfikacja integralności danych, certificate pinning
SSL Pinning Bypass	Pełna	CA w system trust store + root	Twardy pinning (NSC/Frida), certyfikaty użytkownika odrzucone domyślnie (API 24+)

Projekt potwierdza, że ruch HTTPS nie jest bezpieczny, gdy atakujący kontroluje proxy i może zainstalować własne CA na urządzeniu. Kluczową obroną jest weryfikacja certyfikatów po stronie aplikacji (SSL Pinning) oraz integralność danych niezależna od warstwy transportowej.

7.1. Użyte technologie

- **mitmproxy 10** — proxy MITM z interfejsem webowym i skryptami Python,
- **Node.js / Express** — backend HTTPS z TLS 1.2, HSTS i certyfikatem self-signed,
- **Docker / Docker Compose** — izolacja środowiska, sieć `172.20.0.0/24`,
- **Android Studio** — emulator Android 9.0 (API 28), Google APIs, arm64-v8a,
- **adb** — konfiguracja proxy i instalacja certyfikatów na emulatorze.

Spis rysunków

2.1 Działające kontenery wzirni-backend i wzirni-mitm 5

3.1 Uruchomiony emulator Android 9.0 (API 28) 8
3.2 Wyjście skryptu setup.sh zakończone sukcesem 9
3.3 mitmweb UI z pierwszym przechwyconym żądaniem z emulatora 9

4.1 Wyjście demo.sh tls – porównanie nagłówków 11
4.2 mitmweb UI – naglowki odpowiedzi po modyfikacji TLS Downgrade 12

5.1 Wyjście make capture-logs z przechwyconymi danymi 14
5.2 Wyjście demo.sh tamper – porównanie salda 15
5.3 mitmweb UI – żądanie POST /api/login 15
5.4 mitmweb UI – żądanie GET /api/data ze zmodyfikowaną odpowiedzią 16

6.1 Wyjście demo.sh pinning – porównanie certyfikatów 18
6.2 mitmweb UI – TLS handshake i naglowki x-mitm-ssl-bypass 18
6.3 Certyfikat CA mitmproxy w zaufanych certyfikatach systemowych 19

Spis tabel

5.1 Porównanie danych odpowiedzi przed atakiem i po ataku 14

7.1 Zestawienie wyników scenariuszy ataków MITM 20